

Educator Module

```
from google.colab import files
```

```
scn_list = []
print("Upload Scenario 1:")
uploaded = files.upload()
scn_list.append(list(uploaded.keys())[0])

print(scn_list)
```

Upload Scenario 1:

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Save Scenario.txt to Scenario (1).txt

```
import google.generativeai as genai
```

```
# Method to generate PlantUML
```

```
def generate_puml(scenario):
    prompt = (
        f"Generate a valid PlantUML code for the following requirement description: {scenario}. "
        f"Ensure that the PlantUML contains all the correct components specified in the scenario. "
        f"Do not add anything else. Return only the PlantUML code."
    )

    # Configure Gemini API
    api_key = "AIzaSyAvTMTLrbJT06PVb0EQA3lp0rA6KegjqXY"
    genai.configure(api_key=api_key)
    model = genai.GenerativeModel('gemini-2.0-flash')

    response = model.generate_content(
        [{'role': 'user', 'parts': [{'text': prompt}]}],
        generation_config={'temperature': 0.2, 'max_output_tokens': 1024, 'top_k': 10, 'top_p': 0.4}
    )

    return response.text.strip()
```

```
# One scenario file
```

```
scenario_path = '/content/Scenario (1).txt'
```

```
# Read scenario text
```

```
with open(scenario_path, 'r') as file:
    scenario_text = file.read()
```

```
# Generate PlantUML
```

```
puml_code = generate_puml(scenario_text)
```

```
# Save to .puml file
```

```
output_file = 'generated_diagram.puml'
with open(output_file, 'w') as f:
    f.write(puml_code)
```

```
print(f"✅ PlantUML code saved to: {output_file}")
```

✅ PlantUML code saved to: generated_diagram.puml

```
import google.generativeai as genai
```

```
#import base64
```

```
api_key = "AIzaSyAvTMTLrbJT06PVb0EQA3lp0rA6KegjqXY"
genai.configure(api_key=api_key)
model = genai.GenerativeModel('gemini-2.0-flash')
with open("/content/generated_diagram.puml", 'r') as text:
    uml_code = text.read()
prompt = f"From the following PlantUML code, Identify the actors, use cases, actor-to-use case relations, use case-to-use case relations"

'''with open("abstract_factory_uml.png", "rb") as file:
    file_content = file.read()
    encoded_image = base64.b64encode(file_content).decode("utf-8")'''

response = model.generate_content(
    [
        {
            'role': 'user',
            'parts': [{'text': prompt}],
        }
    ],
    generation_config={'temperature': 0.2, 'max_output_tokens': 1024, 'top_k': 10, 'top_p': 0.4}
```

```

)

print(response.text)

{
  "relation_type": "association"
},
{
  "actor": "Librarian",
  "use_case": "Update Book",
  "relation_type": "association"
},
{
  "actor": "Librarian",
  "use_case": "Register Student",
  "relation_type": "association"
},
{
  "actor": "Librarian",
  "use_case": "Remove Account",
  "relation_type": "association"
}
],
"use_case_to_use_case_relations": [
  {
    "use_case_1": "Borrow Book",
    "use_case_2": "Check Availability",
    "relation_type": "include"
  },
  {
    "use_case_1": "Return Book",
    "use_case_2": "Check Availability",
    "relation_type": "include"
  },
  {
    "use_case_1": "Borrow Book",
    "use_case_2": "Update Book Status",
    "relation_type": "include"
  },
  {
    "use_case_1": "Return Book",
    "use_case_2": "Update Book Status",
    "relation_type": "include"
  },
  {
    "use_case_1": "Return Book",
    "use_case_2": "Notify Overdue",
    "relation_type": "extend"
  },
  {
    "use_case_1": "Notify Overdue",
    "use_case_2": "Apply Overdue Fine",
    "relation_type": "extend"
  },
  {
    "use_case_1": "Register Student",
    "use_case_2": "Validate Student Info",
    "relation_type": "include"
  }
],
"actor_to_actor_relations": []
}
...

```

Student Module

```

import requests
import base64
import json

def process_use_case_diagram(api_key, image_path, output_file): # Added output_file as an argument
    # Define the endpoint for the Gemini API
    endpoint = 'https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash-thinking-exp-1219:generateContent?key=' + api_key

    # Read and encode the image
    with open(image_path, "rb") as image_file:
        encoded_image = base64.b64encode(image_file.read()).decode('utf-8')

    # Prepare the request payload
    payload = {
        "contents": [
            {
                "parts": [
                    {
                        "text": "Generate the corresponding PlantUML usecase code for this usecase diagram"
                    },
                    {
                        "inlineData": {

```

```

        "data": encoded_image
    }
}

],
"generationConfig": {
    "temperature": 0.5 # Adjust for creativity
}
}

# Set headers for the request
headers = {
    'Content-Type': 'application/json'
}

# Make the POST request to the Gemini API
response = requests.post(endpoint, headers=headers, data=json.dumps(payload))

# Check for successful response
if response.status_code == 200:
    result = response.json()[ 'candidates' ][0][ 'content' ][ 'parts' ][0][ 'text' ]

    # Save the output to a .puml file
    with open(output_file, "w", encoding="utf-8") as file:
        file.write(result)

    print(f"PlantUML code saved to {output_file}")
else:
    print(f"Error: {response.status_code}, {response.text}")

# Example usage
api_key = 'AIzaSyAgcI_ldwbQpvcacEIUALf1bCrTYbG1vlg' # Replace with your actual API key
image_path = '/content/Result Student.png' # Replace with your image file path
output_file = 'output_use case.puml' # Output file path

process_use_case_diagram(api_key, image_path, output_file) # Calling with 3 arguments now

📄 PlantUML code saved to output_use case.puml

import google.generativeai as genai
#import base64

api_key = "AIzaSyAvTMTLrbJT06PVb0EQa3lpOrA6KegjqXY"
genai.configure(api_key=api_key)
model = genai.GenerativeModel('gemini-2.0-flash')
with open("/content/output_use case.puml", 'r') as text:
    uml_code = text.read()
prompt = f"From the following PlantUML code, Identify the actors, use cases, actor-to-use case relations, use case-to-use case relations"

'''with open("abstract_factory_uml.png", "rb") as file:
    file_content = file.read()
    encoded_image = base64.b64encode(file_content).decode("utf-8")'''

response = model.generate_content(
    [
        { 'role': 'user',
          'parts': [{ 'text': prompt },
                    ]
        },
    ],
    generation_config={ 'temperature': 0.2, 'max_output_tokens': 1024, 'top_k': 10, 'top_p': 0.4 }
)

print(response.text)

📄

```

```

    },
    {
      "actor": "Librarian",
      "use_case": "Remove Book",
      "relation_type": "association"
    },
    {
      "actor": "Librarian",
      "use_case": "Register Student",
      "relation_type": "association"
    },
    {
      "actor": "Librarian",
      "use_case": "Remove Account",
      "relation_type": "association"
    }
  ],
  "use_case_to_use_case_relations": [
    {
      "use_case_1": "Borrow Book",
      "use_case_2": "Check Availability",
      "relation_type": "include"
    },
    {
      "use_case_1": "Return Book",
      "use_case_2": "Check Availability",
      "relation_type": "include"
    },
    {
      "use_case_1": "Return Book",
      "use_case_2": "Notify Overdue",
      "relation_type": "extend"
    },
    {
      "use_case_1": "Notify Overdue",
      "use_case_2": "Apply Fine",
      "relation_type": "include"
    }
  ],
  "actor_to_actor_relations": []
}
...

```

Evaluation Module

```

import requests
import json

# Sample UML diagram dictionaries
diagram1_dict = {
  "actors": [
    "Student",
    "Librarian"
  ],
  "use_cases": [
    "Search Catalog",
    "Borrow Book",
    "Return Book",
    "View Borrowing History",
    "Add Book",
    "Remove Book",
    "Update Book",
    "Register Student",
    "Remove Account",
    "Check Availability",
    "Update Book Status",
    "Notify Overdue",
    "Apply Overdue Fine",
    "Validate Student Info"
  ],
  "actor_to_use_case_relations": [
    {
      "actor": "Student",
      "use_case": "Search Catalog",
      "relation_type": "association"
    },
    {
      "actor": "Student",
      "use_case": "Borrow Book",
      "relation_type": "association"
    },
    {
      "actor": "Student",
      "use_case": "Return Book",
      "relation_type": "association"
    }
  ],
  "actor_to_actor_relations": []
}
...

```

```

    },
    {
      "actor": "Student",
      "use_case": "View Borrowing History",
      "relation_type": "association"
    },
    {
      "actor": "Librarian",
      "use_case": "Add Book",
      "relation_type": "association"
    },
    {
      "actor": "Librarian",
      "use_case": "Remove Book",
      "relation_type": "association"
    },
    {
      "actor": "Librarian",
      "use_case": "Update Book",
      "relation_type": "association"
    },
    {
      "actor": "Librarian",
      "use_case": "Register Student",
      "relation_type": "association"
    },
    {
      "actor": "Librarian",
      "use_case": "Remove Account",
      "relation_type": "association"
    }
  ],
  "use_case_to_use_case_relations": [
    {
      "use_case_1": "Borrow Book",
      "use_case_2": "Check Availability",
      "relation_type": "include"
    },
    {
      "use_case_1": "Return Book",
      "use_case_2": "Check Availability",
      "relation_type": "include"
    },
    {
      "use_case_1": "Borrow Book",
      "use_case_2": "Update Book Status",
      "relation_type": "include"
    },
    {
      "use_case_1": "Return Book",
      "use_case_2": "Update Book Status",
      "relation_type": "include"
    },
    {
      "use_case_1": "Return Book",
      "use_case_2": "Notify Overdue",
      "relation_type": "extend"
    },
    {
      "use_case_1": "Notify Overdue",
      "use_case_2": "Apply Overdue Fine",
      "relation_type": "extend"
    },
    {
      "use_case_1": "Register Student",
      "use_case_2": "Validate Student Info",
      "relation_type": "include"
    }
  ],
  "actor_to_actor_relations": []
}

```

```

diagram2_dict = {
  "actors": [
    "Student",
    "Librarian"
  ],
  "use_cases": [
    "Search Catalog",
    "Borrow Book",
    "Return Book",
    "View Borrowing History",
    "Check Availability",

```

```

    "Notify Overdue",
    "Apply Fine",
    "Add Book",
    "Remove Book",
    "Register Student",
    "Remove Account"
],
"actor_to_use_case_relations": [
    {
        "actor": "Student",
        "use_case": "Search Catalog",
        "relation_type": "association"
    },
    {
        "actor": "Student",
        "use_case": "Borrow Book",
        "relation_type": "association"
    },
    {
        "actor": "Student",
        "use_case": "Return Book",
        "relation_type": "association"
    },
    {
        "actor": "Student",
        "use_case": "View Borrowing History",
        "relation_type": "association"
    },
    {
        "actor": "Librarian",
        "use_case": "Add Book",
        "relation_type": "association"
    },
    {
        "actor": "Librarian",
        "use_case": "Remove Book",
        "relation_type": "association"
    },
    {
        "actor": "Librarian",
        "use_case": "Register Student",
        "relation_type": "association"
    },
    {
        "actor": "Librarian",
        "use_case": "Remove Account",
        "relation_type": "association"
    }
],
"use_case_to_use_case_relations": [
    {
        "use_case_1": "Borrow Book",
        "use_case_2": "Check Availability",
        "relation_type": "include"
    },
    {
        "use_case_1": "Return Book",
        "use_case_2": "Check Availability",
        "relation_type": "include"
    },
    {
        "use_case_1": "Return Book",
        "use_case_2": "Notify Overdue",
        "relation_type": "extend"
    },
    {
        "use_case_1": "Notify Overdue",
        "use_case_2": "Apply Fine",
        "relation_type": "include"
    }
],
"actor_to_actor_relations": []
}

def build_prompt_from_dicts(diagram1, diagram2):
    def format_diagram(diagram):
        actors = "\n".join(f"- {actor}" for actor in diagram["actors"])
        use_cases = "\n".join(f"- {use_case}" for use_case in diagram["use_cases"])
        # Corrected indentation for the next line
        actor_relations = "\n".join(
            f"- '{rel['actor']}' {rel['relation_type']} '{rel['use_case']}'"
            for rel in diagram["actor_to_use_case_relations"]
        )
        use_case_relations = "\n".join(

```

```

f"- '{rel['use_case_1']}' {rel.get('relation_type', 'Unlabeled')} '{rel['use_case_2']}'"
    for rel in diagram.get("use_case_to_use_case_relations", [])
)
actor_actor_relations = ""
if "actor_to_actor_relations" in diagram:
    actor_actor_relations = "\nActor to Actor Relations:\n" + "\n".join(
        f"- '{rel['actor1']}' {rel.get('relation_type', 'Unlabeled')} '{rel['actor2']}'"
        for rel in diagram.get("actor_to_actor_relations", [])
    )

return f"""
Actors:
{actors}

Use Cases:
{use_cases}

Actor to Use Case Relations:
{actor_relations}

Use Case to Use Case Relations:
{use_case_relations}

Actor to Actor Relations:
{actor_actor_relations}
"""

diagram1_text = format_diagram(diagram1)
diagram2_text = format_diagram(diagram2)

# Calculate denominator values from diagram1
actor_count = len(diagram1.get("actors", []))
use_case_count = len(diagram1.get("use_cases", []))
actor_to_use_case_count = len(diagram1.get("actor_to_use_case_relations", []))
use_case_to_use_case_count = len(diagram1.get("use_case_to_use_case_relations", []))
actor_to_actor_count = len(diagram1.get("actor_to_actor_relations", []))

prompt = f"""
Evaluate the similarity of the following two UML use case diagrams represented as structured text.

Use semantic similarity (e.g., "Place Order" vs. "Submit Purchase" or "View Orders" vs. "see Orders") to evaluate the following criteria:
If a particular relationship type (e.g., Use Case-to-Use Case Relation or Actor-to-Actor Relation) is missing in both diagrams, do not
Provide the result in percentage and match count format (X/Y), where Y is the number of relevant elements in Diagram 1 only (the
Use a semantic similarity threshold of 0.7 for determining a valid match between elements.

Output Format (strictly follow this structure):

- Actor Similarity: XX% (X/{actor_count})
- Use-Case Similarity: XX% (X/{use_case_count})
- Actor-to-Use Case Relation Similarity: XX% (X/{actor_to_use_case_count})
- Use Case-to-Use Case Relation Similarity: XX% (X/{use_case_to_use_case_count})
- Actor-to-Actor Relation Similarity: XX% (X/{actor_to_actor_count})
- Overall Similarity: XX% (X/Y)

Compare actor names, use case names, and both types of relationships using semantic similarity. Consider two elements as a
Only provide the scores—no explanations or comments.

Also, give a feedback report identifying missing or mismatched elements in Diagram 2 compared to Diagram 1.

Diagram 1:
{diagram1_text}

Diagram 2:
{diagram2_text}
"""

return prompt

def compare_uuml_dicts(api_key, diagram1_dict, diagram2_dict):
    endpoint = f'https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key={api_key}'

    prompt = build_prompt_from_dicts(diagram1_dict, diagram2_dict)

    payload = {
        "contents": [{"parts": [{"text": prompt}]}],
        "generationConfig": {
            "temperature": 0.3,
            "topK": 1,
            "topP": 1,
            "maxOutputTokens": 1024
        }
    }

```

```

    "maxOutputTokens": 2048
  }
}

headers = {'Content-Type': 'application/json'}

response = requests.post(endpoint, headers=headers, data=json.dumps(payload))

if response.status_code == 200:
    content = response.json()
    parts = content.get("candidates", [])[0].get("content", {}).get("parts", [])
    output_text = "\n".join(part.get("text", "") for part in parts)
    print("\nUML Diagram Comparison (Semantic Similarity):\n")
    print(output_text)
else:
    print(f"Error: {response.status_code}, {response.text}")

# Replace with your actual API key
api_key = "AIzaSyDsMpdj1LjSEfwwf5JW5ZyUSE_l2mQtRKQ"

# Run the comparison
compare_uml_dicts(api_key, diagram1_dict, diagram2_dict)

```



UML Diagram Comparison (Semantic Similarity):

- Actor Similarity: 100% (2/2)
- Use-Case Similarity: 79% (11/14)
- Actor-to-Use Case Relation Similarity: 89% (8/9)
- Use Case-to-Use Case Relation Similarity: 57% (4/7)
- Actor-to-Actor Relation Similarity: 0% (0/0)
- Overall Similarity: 77% (25/32)

Feedback Report:

- Missing Use Cases in Diagram 2: 'Update Book', 'Update Book Status', 'Validate Student Info'
- Mismatched Use Case in Diagram 2: 'Apply Fine' instead of 'Apply Overdue Fine'
- Missing Actor-to-Use Case Relation in Diagram 2: 'Librarian' association 'Update Book'
- Missing Use Case-to-Use Case Relations in Diagram 2: 'Borrow Book' include 'Update Book Status', 'Return Book' include 'Update Book Status'
- Incorrect Use Case-to-Use Case Relations in Diagram 2: 'Notify Overdue' include 'Apply Fine' instead of 'Notify Overdue' extend 'Apply Fine'